

WeepingCAN: A Physical-Layer CAN Simulation Environment for Attack and IDS Modeling

Nicholas Darisi
University of Padua
2196764

Jacopo Falasco
University of Padua
2194529

Filippo Aghi
University of Padua
2200442

I. OBJECTIVES

This project pursues the following interconnected objectives to advance CAN bus security research through comprehensive attack implementation, detection mechanisms, and comparative analysis:

A. First Objective: Bit-Level CAN Bus Simulator

Develop a comprehensive bit-level CAN bus simulator in Python that accurately reproduces the Controller Area Network protocol's core mechanisms. Unlike existing frame-level commercial tools, our simulator enables granular control and observation at the bit level, supporting:

- 1) Accurate CAN Frame Construction with automatic bit stuffing, CRC-15/CAN computation, and all ISO 11898-1 standard fields.
- 2) Priority-Based Bus Arbitration using Wired-AND semantics, simulating the physical media access mechanism where lower CAN IDs win arbitration through bit-by-bit comparison.
- 3) Error Detection and Handling with proper TEC/REC counter management, state transitions, and generation of active/passive error flags.
- 4) Multi-threaded ECU Node Simulation with periodic message transmission, realistic payload generation, and configurable transmission periods.

B. Second Objective: Bit-Level CAN Bus Simulator

Implement the bus-off attack as described by Cho and Shin, demonstrating how a malicious ECU can force legitimate in-vehicle nodes off the network:

- 1) Attack Mechanism: leverage CAN's fault confinement mechanism to accumulate the Transmit Error Counter (TEC) in victim nodes by injecting collisions at precise synchronization points.
- 2) Implementation Details:
 - Monitor victim ECU transmission timing patterns
 - Synchronize attacker frame submission with victim message transmission windows
 - Craft messages with same arbitration ID to cause post-arbitration collisions
 - Force the generation of active error flags in victim ECU, incrementing TEC by 8 per collision
- 3) Success Criterion: force victim node into bus-off state.

C. Third Objective: WeepingCAN Stealthy Attack Implementation

Implement the WeepingCAN attack as a stealthier variant of classical bus-off that evades detection through sophisticated techniques:

1) Stealthiness Features:

- Disable Retransmission: prevent transmission of automatic retransmit frames that would increase observability.
- Recessive Bit Injection: instead of causing detectable dominant-bit collisions, attack via recessive-to-dominant mismatches.
- Payload Mimicry: learn and predict victim payload patterns to send nearly-identical frames, reducing collision signatures.

2) Multi-Phase Attack Methodology: passively learn victim timing and payload patterns, then synchronously inject collisions on predicted-zero bits to stealthily raise TEC without detection.

3) Key Differentiators from Classical Bus-Off:

- Collision Signature: reduced frequency and obviousness.
- TEC Growth: slower, irregular pattern vs. linear progression (harder to detect with simple thresholds).

D. Fourth Objective: Intrusion Detection System (IDS) Development

Design and implement a two-level IDS capable of detecting both standard bus-off attacks and stealthy WeepingCAN-style intrusions:

1) Information Theory Level

- Detects low-entropy or highly concentrated bit patterns indicative of abnormal activity.
- Identifies the most frequent offender to highlight a likely attacker.

2) Traffic Observation Level

- Continuously monitors steady-state behavior to capture emerging attack patterns.
- Estimates attacker TEC by tracking and interpreting full-bus traffic dynamics.

II. SYSTEM SETUP: CUSTOM CAN BUS SIMULATOR

To support the analysis of the attacks, a custom CAN bus simulator was developed in Python. The simulator was designed specifically for this study to provide fine-grained visibility over arbitration, bit-level frame evolution, collision dynamics, and error-state transitions. This level of detail was necessary because the attack scenarios considered in this work do not depend solely on message-level interactions, but on the precise evolution of individual bits on the bus and on the resulting behavior of the CAN fault confinement mechanism.

The simulator implements classical CAN communication with 11-bit identifiers and models the main protocol mechanisms required for attack-oriented experimentation, including frame serialization, bit stuffing, CRC generation, wired-AND arbitration, same-identifier collisions, transmission error detection, and node transitions across the error active, error passive, and bus off states. Rather than relying on a message-level abstraction, the platform was designed to explicitly expose how protocol compliant transmissions interact on the shared medium and how repeated bit-level conflicts can progressively isolate a target ECU from the network.

A. Architecture Overview

The simulator follows a master-slave architecture centered on a software bus controller, denoted as CANBusMaster, which coordinates multiple Electronic Control Units (ECUs). The bus master acts as the centralized arbitration entity and is responsible for collecting transmission requests, performing bit-by-bit arbitration, detecting collisions, generating error conditions, and updating the global execution state used for runtime observation.

Legitimate network nodes are modeled through the BaseECU abstraction. Each ECU periodically generates payloads, submits transmission requests to the bus master, maintains local Transmit Error Counter (TEC) and Receive Error Counter (REC) values, and updates its communication state according to the standard CAN fault confinement rules. This design makes it possible to reproduce node-level behavior under both normal communication and adversarial conditions.

Frame generation is handled by the CANBitStream component, which serializes CAN frames at bit level. This module encodes all protocol fields, computes the CRC-15/CAN checksum, and applies bit stuffing over the relevant frame portion. The resulting modular organization separates bus arbitration, node behavior, and frame construction while preserving full visibility over every transmitted bit. Such visibility is central to this work, since the attack logic depends on the exact moment at which two simultaneous transmissions diverge and on the way this divergence is interpreted by the transmitter.

B. Bit-Level Frame Construction

The simulator generates base-format CAN frames according to the ISO 11898-1 standard ordering of protocol fields.

All fields are serialized explicitly at bit level. Identifiers are transmitted most significant bit first, and payload bytes are likewise emitted from the most significant bit to the least

significant bit. This choice allows the simulator to preserve the exact temporal structure of the frame and to represent not only which message is transmitted, but how its individual bits appear on the bus.

This level of representation is especially important for the experimental goals of this work: in the considered attack scenarios, the relevant event is not merely that two frames coexist, but that they remain aligned through arbitration and subsequently diverge at a specific non-arbitration bit. A frame-level representation would not be sufficient to capture this behavior with the required precision.

C. Bit Stuffing and CRC Generation

The CANBitStream module implements bit stuffing from the SOF up to and including the CRC field. After five consecutive bits of identical polarity, a complementary bit is inserted automatically. Stuffing is implemented through a run-length tracking mechanism that monitors the current bit value and the number of consecutive repetitions already emitted. Once the run length reaches five, a complementary stuff bit is inserted and explicitly marked as such in the internal frame representation.

Modeling bit stuffing is essential for two reasons. First, it preserves the synchronization semantics expected in CAN communication. Second, it alters the actual position of subsequent bits on the bus. Since the attack scenarios considered in this work depend on predicting and exploiting divergence points between concurrent transmissions, the dynamic insertion of stuffed bits becomes part of the effective attack surface and cannot be abstracted away without losing fidelity.

The simulator also computes the CRC-15/CAN checksum using the standard polynomial 0x4599. The CRC is generated over the unstuffed frame content from SOF through the end of the data field and appended before delimiter and acknowledgment bits. Explicit CRC generation ensures that legitimate transmissions produced by the simulator are internally consistent and protocol-conformant. Although complete receiver-side CRC validation is not the main focus of the attack logic, including CRC generation contributes to preserving the structural realism of the transmitted frames.

D. Arbitration Model

Transmission requests from ECUs are collected by the bus master through a shared pending queue protected by a synchronization lock. At each collection cycle, the bus master atomically acquires the current set of pending transmissions and processes them through the arbitration routine. This design avoids race conditions while enabling concurrent transmission attempts from multiple ECUs.

A deliberate admission policy is adopted at queue level: each ECU may have at most one pending frame at a time: if a node attempts to submit a new frame while another transmission from the same ECU is already pending, the new request is discarded. Although simplified with respect to real controller buffering behavior, this choice improves timing determinism and limits uncontrolled retransmission effects.

Such determinism is particularly useful in the present context because the experimental analysis focuses on adversarial synchronization with victim transmissions.

Arbitration itself is performed bit by bit according to the wired-AND semantics of CAN. During the arbitration phase, which includes SOF, identifier, and RTR, every contender compares the observed bus value with the bit it attempted to transmit. If a node transmits recessive and observes dominant, it loses arbitration and is removed from the active contender set. When a unique winner emerges, that ECU completes frame transmission normally and the resulting message is recorded as the last successful transmission on the bus. This faithfully reproduces the standard CAN priority mechanism, under which numerically lower identifiers dominate higher ones.

E. Same-Identifier Collisions and Error Propagation

One of the main features of the simulator is the explicit handling of concurrent transmissions that share the same arbitration identifier. If multiple contenders remain active beyond the arbitration portion of the frame, the simulator interprets the event as a same-identifier collision and transfers execution to a dedicated collision-handling routine. At that point, the competing frames are still indistinguishable at protocol level from the perspective of arbitration, and the bus must be observed during the subsequent control and data fields to determine whether a transmission error arises.

The collision handler continues processing the concurrent transmissions bit by bit and monitors for bit-monitoring errors. A transmission error is detected when a node sends a recessive bit but observes a dominant value on the bus outside the arbitration fields.

This mechanism is central to the study of bus-off attacks. It captures the exact protocol-level condition under which a malicious ECU can force a victim to detect a transmission error while still operating within the ordinary semantics of CAN signaling. The simulator therefore makes it possible to study not only whether a collision occurs, but also how that collision is interpreted by the transmitting nodes and how it propagates into error-state evolution.

F. Fault Confinement and Node State Evolution

Each ECU maintains the standard CAN fault confinement variables, namely the Transmit Error Counter and the Receive Error Counter, together with an internal communication state. When a collision-induced transmission error is detected, the simulator updates node counters according to the modeled error semantics and emits the corresponding error flag. If the offending ECU is in the error active state, an active error flag is generated; if it is in the error passive state, a passive flag is emitted instead. Successful transmissions reduce the transmitter TEC by one, down to a minimum of zero, and successful receptions may similarly reduce previously accumulated REC values.

By reproducing the full evolution from a state to a different one, the simulator provides the necessary foundation for ana-

lyzing how repeated same-ID conflicts can isolate a legitimate ECU from the bus.

G. Timing Model and Experimental Determinism

The simulator uses a multi-threaded execution model in which the bus master runs in a dedicated background thread and each ECU executes its own transmission loop concurrently. This architecture allows multiple nodes to behave independently while preserving centralized arbitration and collision resolution.

Each ECU maintains a `next_tx_time` variable representing the next scheduled transmission instant. After each submission, this value is advanced by the node transmission period, and if the current time has already surpassed the scheduled instant, it is incremented again until it falls in the future. This mechanism avoids timing drift and produces stable periodic transmission behavior over long runs.

The arbitration and collision routines also include deliberate per-bit delays to make bus activity observable through the runtime interface. These delays are not intended to replicate real physical bus timing. Instead, they provide a controlled and interpretable experimental environment in which arbitration outcomes, collisions, and error propagation can be inspected step by step. From the perspective of this work, this represents a deliberate trade-off between physical timing realism and analytical visibility.

III. CLASSICAL BUS-OFF ATTACK IMPLEMENTATION

The classical bus-off attack was implemented by exploiting the CAN fault confinement mechanism through repeated transmission errors intentionally induced on a target ECU. The attack model assumes the presence of a malicious node capable of transmitting on the bus with the same arbitration identifier as the victim and with sufficient timing accuracy to overlap the victim's transmission attempts.

In the implemented scenario, the attacker ECU was configured to use the same arbitration ID as the target ECU. This prevents arbitration from resolving the contention during the identifier field, since both nodes transmit identical bits throughout the arbitration phase. As a result, both frames remain active beyond arbitration and continue transmission concurrently into the control and data fields. The attack is then triggered by forcing a bit mismatch in the non-arbitration portion of the frame, so that the victim transmits a recessive bit while the attacker drives the bus to a dominant state. According to the CAN bit-monitoring rule, this condition is interpreted as a transmission error by the victim.

Each induced bit-monitoring error increments the victim's Transmit Error Counter (TEC), thereby activating the standard fault confinement process. By repeating the collision process over successive transmission attempts, the attack progressively drives the victim from the error active state to error passive state, and ultimately to bus off once the TEC threshold is reached. In the bus off state, the target ECU is no longer allowed to transmit.

A relevant aspect of the implemented attack emerges once the victim reaches the error passive state. In this condition, when the victim detects a transmission error, it no longer emits an active error flag, but a passive error flag. Owing to the wired-AND semantics of the bus, even a single dominant bit is enough to overwrite the corresponding recessive bit sent by the victim. As a result, the victim’s passive error signaling no longer forces a visible dominant error condition on the bus and does not effectively prevent the attacker from completing its transmission.

This behavior has an important consequence for the long-term dynamics of the attack: while the victim continues to detect bit-monitoring errors and therefore keeps incrementing its TEC, the attacker can successfully complete its own transmission in the presence of the victim’s passive error flag. Consequently, once the victim has entered error passive, the attack becomes even more favorable to the adversarial node, because the victim loses the ability to actively corrupt the bus with dominant error flags, while the attacker remains able to impose the bus state. This mechanism explains why, in the later stages of the attack, the victim is driven steadily toward bus off while the attacker tends to remain in error active state.

From an implementation perspective, the attack did not require modifications to the arbitration logic of the simulator or to the CAN error state machine. Instead, it was realized as a malicious transmission strategy at ECU level. More specifically, the attacker was configured to (i) impersonate the victim’s arbitration identifier, (ii) synchronize its transmission timing with the target, and (iii) transmit a frame whose data field diverges from that of the victim so as to provoke a bit-monitoring error after arbitration. The attack therefore remains protocol-compliant in the sense that it leverages standard CAN transmission and fault confinement rules rather than relying on external channel jamming or non-CAN signaling behavior.

This baseline implementation provides the reference fault-confinement attack model upon which WeepingCAN is built. Starting from the same same-ID collision principle, WeepingCAN extends the classical bus-off strategy with victim-aware synchronization, probabilistic payload modeling, and attacker-side TEC regulation.

IV. WEEPINGCAN ATTACK IMPLEMENTATION

The WeepingCAN attack was implemented as an adaptive, timing-aware extension of the classical bus-off attack: it augments the previous one with a preparatory learning phase aimed at improving both synchronization accuracy and collision effectiveness. In the implemented design, the attacker does not immediately inject malicious frames. Instead, it first observes bus traffic, identifies a suitable victim, estimates its transmission behavior, derives a probabilistic model of the victim payload, and only then starts transmitting colliding frames. This makes the attack more selective and significantly more effective than a purely brute-force strategy.

A. Traffic Observation and Victim Selection

In the initial stage, the attacker remains passive and records bus traffic over a configurable observation window. For each captured frame, it stores the arbitration identifier, payload bytes, bit-level representation, and timestamp, thereby building a traffic profile organized per identifier.

At the end of this phase, the attacker selects as victim the arbitration ID associated with the highest number of observed transmissions. This heuristic identifies the most active ECU on the bus and provides both a larger observation base and more frequent attack opportunities.

B. Victim Timing Estimation and Synchronization

After selecting the victim, the attacker estimates its transmission timing. If the victim ECU is directly accessible within the simulator, the attacker uses the configured transmission period and next scheduled transmission instant as synchronization parameters. Otherwise, it infers the victim period from the inter-arrival times of previously sniffed frames.

The resulting timing model is then used to align the malicious transmission with the victim’s next transmission attempt. By injecting the attack frame in the same transmission window, the attacker increases the probability of surviving arbitration together with the victim and reaching the non-arbitration portion of the frame concurrently.

C. Bit-Level Profiling of the Victim Payload

A central component of WeepingCAN is the bit-level analysis of the victim payload. From the frames collected during the first sniffing phase, the attacker computes, for each payload bit position, the empirical probability of observing a dominant value or a recessive value. This produces a probabilistic representation of the victim traffic that captures the degree of stability associated with each individual bit.

The purpose of this step is not merely to identify predictable fields in a generic sense, but to determine which payload positions exhibit a strong bias toward a dominant bus value. These positions are particularly relevant for the attack, since they provide the attacker with candidate divergence points that can be exploited during the collision phase.

D. Prediction Validation on Reinforcement Traces

After the initial profiling stage, the attacker performs a second victim-focused sniffing phase. The frames collected in this interval are not used to rebuild the model from scratch, but rather to validate the consistency of the bit-level statistics extracted from the first observation window.

Concretely, for each analyzed bit position, the attacker compares the most likely value inferred from the initial dataset with the actual value observed in the reinforcement traces and computes an overall prediction accuracy. This phase therefore acts as a validation step: it confirms whether the learned bit tendencies remain sufficiently stable over time to support the subsequent attack stage.

E. Sequential Payload Prediction and Bit Selection

Once the victim bit model has been validated, the attacker estimates the next victim payload by exploiting the recent transmission history.

Starting from the resulting predicted payload, the attacker identifies the candidate bit positions that are expected to be dominant in the next victim frame. Each candidate is then assigned a score that combines the empirical probability of being dominant with a penalty term that discourages repeated use of the same bit across successive attack attempts. The final target bit is selected probabilistically among the top-ranked candidates through an exploration–exploitation policy based on epsilon-randomization and softmax sampling.

After selection, only that bit is flipped, changing it from dominant to recessive in the attacker frame. The malicious payload therefore remains as close as possible to the predicted victim payload, while introducing a single controlled divergence at a strategically chosen position.

F. Collision Induction in the Data Field

During the attack phase, the malicious ECU transmits with the same arbitration identifier as the victim. Since both nodes transmit identical identifier bits, arbitration cannot resolve the contention and both frames remain active beyond the arbitration phase.

The crafted attacker payload differs from the predicted victim payload in exactly one selected bit, which is forced to recessive even though the victim is expected to transmit it as dominant with high probability. Under the wired-AND semantics of CAN, the resulting bus value remains dominant. Consequently, the collision is converted into a bit-monitoring event in the data field, which activates the CAN error-handling mechanism.

When the attacker is in the error active state, the resulting active error flag causes the current transmission to be aborted for all involved nodes, and both attacker and victim accumulate transmit-side error penalties. The effectiveness of the attack therefore depends on generating these collisions persistently while keeping the attacker below its own critical TEC region.

G. Attacker Self-Preservation and TEC Regulation

A key feature of the implementation is the explicit regulation of the attacker’s own Transmit Error Counter. Since the attack deliberately triggers repeated same-ID collisions, the attacker would otherwise risk entering error passive before the victim. To prevent this, WeepingCAN continuously monitors its local TEC and modulates its aggressiveness accordingly.

The attack probability is reduced as the attacker TEC increases, so that collision attempts become less frequent in riskier conditions. When the TEC exceeds a hard threshold, the attacker temporarily suspends the offensive phase and enters a cooldown mode. In this state, it transmits auxiliary non-conflicting frames with high-priority identifiers and random payloads in order to obtain successful transmissions and reduce its TEC. Additional cleanup transmissions may also

be injected after an attack attempt, with the number of such reset frames increasing as the attacker TEC grows.

This self-preservation mechanism is a defining difference with respect to the classical baseline. In the baseline bus-off attack, repeated collisions tend to push both nodes toward fault confinement. In WeepingCAN, by contrast, collision induction is coupled with an explicit TEC recovery strategy, allowing the attacker to remain operational for longer periods and to sustain the attack until the victim reaches the bus off state.

V. IDS IMPLEMENTATION

The IDS is designed as a self-protection mechanism embedded within each ECU, rather than as an external observer, enabling direct and localized defense capabilities. By leveraging internal knowledge of its own fault-confinement parameters, each node can react autonomously to abnormal bus behavior. The aim was to define a distributed system of anomaly observers, with each ECU acting as an autonomous protection node rather than relying on a centralized supervisory unit.

A. Information Theory Level

The first layer analyzes the statistical properties of the observed bitstream to uncover deviations from normal CAN behavior. It evaluates entropy and bit pattern concentration to detect whether errors cluster around specific bit positions or exhibit unusually low randomness; both are strong indicators of deliberate interference rather than natural noise. By tracking the frequency of offenders associated with these anomalies, it highlights the node most likely responsible for injecting disturbances, providing an early and lightweight intrusion signal before deeper analysis is required.

B. Traffic Observation Level

This layer is designed to operate as a real-time monitoring thread that continuously inspects bus activity. A dedicated state machine structures the observation process into distinct phases, allowing the IDS to track how the attack unfolds over time. By correlating these phases, the system reconstructs the evolving intrusion pattern, enabling the detection of both the attacker and the affected ECU while maintaining full awareness of the bus’s dynamic behavior.

The observer operates through a sequence of four finite phases:

- victim identification
- target monitoring
- attacker inference
- bus tracking and pattern recognition

Each phase, implemented within a dedicated state machine, is designed to exploit specific vulnerabilities in the attacker’s procedure.

C. WeepingCAN Vulnerabilities

The WeepingCAN attack relies on specific heuristics to optimize the standard Bus-Off strategy while remaining stealthy, by having a similar behavior of a common physical layer fault.

Two characteristic behaviors emerge and form the basis for designing an effective IDS.

When disturbing dominant bits, a malicious ECU risks entering the Error Passive state, which would make the attack ineffective. As discussed, this condition is avoided by periodically reducing the attacker's TEC through legitimate frame transmissions during a cooldown phase. The attacker may also transmit frames during the attack window, but to maintain a low profile it must adopt a very low transmission rate, often insufficient to maintain the Error Active state.

Once the victim crosses the Error Passive threshold, the attacker has effectively completed its task: the victim's TEC begins to rise, while the attacker's TEC decreases. These two complementary behaviors form a recognizable pattern that can be systematically observed and exploited for intrusion detection.

D. Victim Identification

In this first phase, the observer scans the ongoing traffic to detect which frame ID is showing abnormal error growth. The logic monitors frame outcomes, updates local counters, and flags the IDs whose behavior is suspicious. Once a consistent rise in the suspected victim's TEC is observed, the IDS locks the target and moves to the next phase.

E. Target Monitoring

After identifying the potential victim, the observer focuses on tracking its behavior over a short, controlled window. The goal is to characterize the transmission rate of all messages within the error progression sliding window. The code continuously samples the bus and verifies that the victim's TEC trend remains coherent with a real attack scenario. If the pattern stabilizes, the observer proceeds; otherwise, it resets and returns to the initial state.

F. Attacker Inference

Once the victim is confirmed, the observer shifts to identifying the attacker. In this phase, the IDS evaluates which identifiers are increasing their transmission rate while the victim TEC decreases, inferring a cooldown phase on the attacker's side. By comparing the monitored traffic with the observed rate increments, the system decides whether to proceed to the final phase or, alternatively, whether the behavior is still compatible with a general bus fault, requiring a return to the victim identification phase.

G. Bus Tracking and Pattern Recognition

In the last phase, the observer tracks the full interaction between victim and attacker. The code monitors the TEC evolution of both nodes and validates that the sequence matches the known WeepingCAN pattern. This continuous tracking allows the IDS to confirm the attack, record its progression, and maintain awareness of the bus condition until the pattern ends or the system resets.

VI. EXPERIMENTS

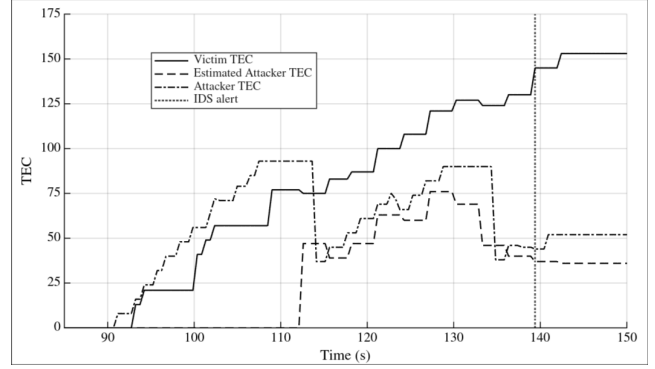


Fig. 1. IDS analyzed behavior

The analysis of the simulated attack set shows a generally good alignment between the estimated and actual attacker TEC, with small mismatches that remain within an acceptable range for this type of lightweight inference (Fig.1).

The main limitations emerge in scenarios where the attack is fragmented across multiple IDs with highly unbalanced transmission rates, which naturally reduces estimation capability. Despite this, the overall results remain positive.

<https://github.com/Cobr3x/weepingCAN-ml-attack.git>

VII. RESULTS AND DISCUSSION

The physical-layer simulation behaves as expected, and the overall architecture performs correctly. The first IDS layer consistently detects Bus-Off attacks, while WeepingCAN attacks require the second layer to reach a correct decision.

The IDS has been implemented in the simulation as a single observer instance; however, the underlying logic remains fully compliant with the distributed approach discussed earlier. This choice is a practical simplification: in the simulated CAN bus, each ECU transmits only one identifier, and the attacked ID directly corresponds to the ECU's own TEC evolution. Under these conditions, a single instance of the observer can track both the victim and infer the attacker, since the ID error counter is equivalent to the ECU TEC. In a real distributed deployment, each ECU would run its own local observer, using its internal knowledge of fault confinement parameters and managing multiple transmitted IDs.

The main bottleneck of the system lies in the attacker's IDs inference. An attacker could use multiple identifiers to inject faults, making the mapping between IDs and the attacker's TEC less direct. In such cases, the inference mechanism would require a more robust strategy to estimate the attacker's TEC. The larger the mismatch between the inferred TEC and the real attacker behavior, the lower the IDS capability to recognize the characteristic WeepingCAN pattern.

Moreover the large number of parameters required to properly tune the IDS, such as the observer windows, highlights the need for a machine learning based approach, capable of adapting to different channel baud rates and to the transmission patterns of both attacker and legitimate ECUs.